

SPAQ-DL-SLAM: Towards Optimizing Deep Learning-based SLAM for Resource-Constrained Embedded Platforms

Niraj Pudasaini¹, Muhammad Abdullah Hanif¹, Muhammad Shafique¹

Abstract—Optimizing Deep Learning-based Simultaneous Localization and Mapping (DL-SLAM) algorithms is essential for efficient implementation on resource-constrained embedded platforms, enabling real-time on-board computation in autonomous mobile robots. This paper presents SPAQ-DL-SLAM, a framework that strategically applies Structured Pruning and Quantization (SPAQ) to the architecture of one of the state-of-the-art DL-SLAM algorithms, DROID-SLAM, for resource and energy-efficiency. Specifically, we perform structured pruning with fine-tuning based on layer-wise sensitivity analysis followed by 8-bit post-training static quantization (PTQ) on the deep learning modules within DROID-SLAM. Our SPAQ-DROID-SLAM model, optimized version of DROID-SLAM model using our SPAQ-DL-SLAM framework with 20% structured pruning and 8-bit PTQ, achieves an 18.9% reduction in FLOPs and a 79.8% reduction in overall model size compared to the DROID-SLAM model. Our evaluations on the TUM-RGBD benchmark shows that SPAQ-DROID-SLAM model surpasses the DROID-SLAM model by an average of 10.5% on absolute trajectory error (ATE) metric. Additionally, our results on the ETH3D SLAM training benchmark demonstrate enhanced generalization capabilities of the SPAQ-DROID-SLAM model, seen by a higher Area Under the Curve (AUC) score and success in 2 additional data sequences compared to the DROID-SLAM model. Despite these improvements, the model exhibits performance variance on the distinct Vicon Room sequences from the EuRoC dataset, which are captured at high angular velocities. This varying performance at some distinct scenarios suggests that designing DL-SLAM algorithms taking operating environments and tasks in consideration can achieve optimal performance and resource efficiency for deployment in resource-constrained embedded platforms.

I. INTRODUCTION

Simultaneous Localization and Mapping (SLAM) is a technique that enables an agent to map an unknown environment while simultaneously tracking its location. SLAM is used for robotic autonomy and is an intrinsic component of embodied AI systems for navigation [1] [2]. As embodied agents are increasingly deployed in complex, dynamic real-world scenarios, the demand for accurate, robust, yet computationally efficient onboard SLAM systems has grown significantly [3].

Current SLAM approaches can be categorized into classical and deep learning-based methods. Classical SLAM, exemplified by ORB-SLAM3 [4], relies on handcrafted feature detection and matching algorithms. While computationally efficient, these methods often struggle in complex, dynamic environments [5]. In contrast, DL-SLAM systems, such as DROID-SLAM [6], Rover-SLAM [7], GO-SLAM [8], and

DVI-SLAM [9], integrate deep neural networks (DNNs) to enhance the perception and interpretation of the environment, leading to significant improvements in accuracy, robustness, and generalization. Specifically, DROID-SLAM shows a 43% reduction in error over ORB-SLAM3 on the EuRoC dataset and an 83% reduction on the TUM-RGBD dataset for the sequences where ORB-SLAM3 is successful [6].

However, the computational demands of DL-SLAM systems (see Table I) pose a significant challenge for on-board computation in mobile robots. These requirements often exceed the processing capabilities of embedded platforms commonly used in mobile robots (see Table II).

TABLE I
HARDWARE CONFIGURATIONS USED IN TESTING DL-SLAM SYSTEMS

DL-SLAM System	GPU Model (CUDA Cores)	Memory Size (GB)
DROID-SLAM [6]	2 × NVIDIA RTX 3090 (2 × 10,496)	2 × 24 (GDDR6X)
GO-SLAM [8]	NVIDIA RTX 3090 (10,496)	24 (GDDR6X)
Rover-SLAM [7]	NVIDIA RTX 3080 (8,704)	10 or 12 (GDDR6X)
DVI-SLAM [9]	NVIDIA RTX 3090 (10,496)	24 (GDDR6X)

For example, the real-time implementation of DROID-SLAM requires two NVIDIA RTX 3090 GPUs, each equipped with 10,496 CUDA cores (see Table I). The system divides its workload, running front-end tracking and local bundle adjustment on the first GPU, while the second GPU handles back-end global bundle adjustment and loop closure, requiring a total memory of around 24 GB [6]. In contrast, one of the most powerful embedded platforms in mobile robotics, the NVIDIA Jetson AGX Orin, has only 2,048 CUDA cores—just 9.76% of what DROID-SLAM necessitates—and significantly less memory capacity (see Table II). This disparity significantly challenges the deployment of state-of-the-art DL-SLAM algorithms on embedded platforms for mobile robots.

TABLE II
COMPUTATIONAL AND BATTERY CAPACITY OF COMMON MOBILE ROBOTS IN ROBOTICS R&D

Robot (Category)	GPU Model (CUDA Cores)	Memory / Battery Capacity
NVIDIA JetBot (UGV)	NVIDIA Jetson Nano (128)	4 GB / 10,000 mAh Li-Po
Skydio 2 (Drone)	NVIDIA Jetson TX2 (256)	8 GB / 4,280 mAh
Boston Dynamics Spot (Legged)	NVIDIA Jetson Xavier (512)	16 GB / 605 Wh
Clearpath Robotics Husky (UGV)	NVIDIA Jetson AGX Orin (2048)	32 GB / 2x 10Ah, 24V Li-Ion

Our Novel Contributions: This paper presents a resource and energy-efficient optimization framework SPAQ-DL-SLAM (see Fig 1), applied to the architecture of DROID-SLAM. To the best of our knowledge, this is the first

¹ All authors are affiliated with [New York University](#) (NYUAD), UAE {np2289, mh6117, muhammad.shafique}@nyu.edu